

Pytest for Flask Application

This is the documentation for how to test APIs in a flask application using pytest module.

It contains brief about the setup and project structure, along with sample code snippets and commands to execute the tests.

It will also contain some basic definitions of important terms.

Structure

The structure should ideally be as follows:

tests --

- | [_conftest.py](#)
- | [_test_routes.py](#)
- | [__init__.py](#)

The 'tests' directory should be at the same level of the folder/ directory where the app is initialized/ instantiated.

For eg,

- apps-|
 - | [_content_generator](#) (contains helper, router, [__init__.py](#) file where app is initialized)
 - | [_tests](#)
 - | [__init__.py](#)
 - | [_app.py](#)

Fixtures

Pytest *fixtures* allow writing pieces of code that are reusable across tests. A simple fixture returns a value, but a fixture can also do setup, yield a value, then do teardown.

If you're using an [application factory](#), define an `app` fixture to create and configure an app instance. If you're not using a factory, you already have an app object you can import and configure directly.

```
#apps/test/conftest.py
@pytest.fixture()
def base_url():
    return "http://localhost/3000"
```

This sets up a base url for all tests that have 'base_url' passed as an argument to them.

for eg;

When writing a test case, pass 'base_url' as a parameter

```
def test_hello(base_url):  
    pass
```

Pytest uses fixtures by matching their function names with the names of arguments in the test functions. For example, the `test_hello` function you'll write next takes a `client` argument. Pytest matches that with the `client` fixture function, calls it, and passes the returned value to the test function.

Initializing and Importing App

Use `'..'` as needed, if relative imports are required.

```
import pytest  
from ..root_app import app as test_app  
import logging  
  
@pytest.fixture()  
def app():  
    app = test_app  
    app.config.update({  
        "TESTING": True,  
    })  
    yield app  
  
@pytest.fixture()  
def client(app):  
    return app.test_client()
```

test_client(): Creates a test client for this application. For information about unit testing head over to [Testing Flask Applications](#).

Note that if you are testing for assertions or exceptions in your application code, you must set `app.testing = True` in order for the exceptions to propagate to the test client. Otherwise, the exception will be handled by the application (not visible to the test client) and the only indication of an `AssertionError` or other exception will be a 500 status code response to the test client.

The `client` fixture calls `app.test_client()` with the application object created by the `app` fixture. Tests will use the client to make requests to the application without running the server.

Test Cases:

```
#apps/tests/test_routes.py  
def test_index(client, caplog, caplog_level):  
    with caplog.at_level(caplog_level):  
        response = client.get("/")  
        assert response.status_code == 200
```

```
def test_health_check(client):
    response = client.get("/health-check", headers={'x': 'y'})
    assert response.status_code == 200
    assert b"Welcome" in response.data
```

Testing Process

The test client makes requests to the application without running a live server.

The `client` has methods that match the common HTTP request methods, such as `client.get()` and `client.post()`. They take many arguments for building the request; you can find the full documentation in [EnvironBuilder](#). Typically you'll use `path`, `query_string`, `headers`, and `data` or `json`.

Pass a dict `query_string={"key": "value", ...}` to set arguments in the query string (after the `?` in the URL). Pass a dict `headers={}` to set request headers.

To send a request body in a POST or PUT request, pass a value to `data`. If raw bytes are passed, that exact body is used. Usually, you'll pass a dict to set form data.

Running Tests

Run the command 'pytest'

Or with full path and filename:

```
pytest apps/tests/test_routes.py
```



If any tests fail, pytest will show the error that was raised. You can run `pytest -v` to get a list of each test function rather than dots.



To measure the code coverage of your tests, use the `coverage` command to run pytest instead of running it directly.

Path

When you use the `test_client` in Flask, the code simulates HTTP requests internally within your application context. It doesn't involve actual network communication, so there's no need to specify or even consider a port. The Flask app handles routing based on the provided paths, as if they were part of a full URL, but without ever needing to bind to a port.

Here's how it works:

1. **Base URL Assumption:** Flask assumes a base URL of `http://localhost/` for the test client. When you provide just the endpoint, such as `"/health-check"`, Flask combines it with the base URL to form the full URL (`http://localhost/health-check`).
2. **Routing in Flask:** Flask's routing system maps this URL path (`"/health-check"`) to the appropriate view function in your application. Since you've already configured your Flask app within the `app` fixture and passed it to the `test_client`, Flask knows how to route the request to the correct view function.
3. **Testing Environment:** In the testing environment (indicated by `"TESTING": True` in the config), Flask is in a special mode where it doesn't need to serve the application over a real network. The `test_client` simulates HTTP requests within the application context, meaning it directly interacts with the Flask routing system to find the correct handler for `"/health-check"`.

Coverage

Test coverage is defined as a technique which determines whether our test cases are actually covering the application code and how much code is exercised when we run those test cases.

Types of Coverage:

- **Function coverage:** how many of the functions defined have been called.
- **Statement coverage:** how many of the statements in the program have been executed.
- **Branches coverage:** how many of the branches of the control structures (if statements for instance) have been executed.
- **Condition coverage:** how many of the boolean sub-expressions have been tested for a true and a false value.
- **Line coverage:** how many of lines of source code have been tested.

To measure the code coverage of your tests, use the `coverage` command to run pytest instead of running it directly.

```
coverage run -m pytest
```

You can either view a simple coverage report in the terminal:

```
coverage report
```

An HTML report allows you to see which lines were covered in each file:

```
coverage html
```

This generates files in the `htmlcov` directory. Open `htmlcov/index.html` in your browser to see the report.



Skipping And Failing Tests:

A **skip** means that you expect your test to pass only if some conditions are met, otherwise pytest should skip running the test altogether. Common examples are skipping windows-only tests on non-windows platforms, or skipping tests that depend on an external resource which is not available at the moment (for example a database).

The simplest way to skip a test function is to mark it with the `skip` decorator which may be passed an optional `reason`:

```
@pytest.mark.skip(reason="no way of currently testing this")
def test_the_unknown(): ...
```

Alternatively, it is also possible to skip imperatively during test execution or setup by calling the `pytest.skip(reason)` function:

```
def test_function():
    if not valid_config():
        pytest.skip("unsupported configuration")
```

An **xfail** means that you expect a test to fail for some reason. A common example is a test for a feature not yet implemented, or a bug not yet fixed. When a test passes despite being expected to fail (marked with `pytest.mark.xfail`), it's an **xpass** and will be reported in the test summary.

You can use the `xfail` marker to indicate that you expect a test to fail:

```
@pytest.mark.xfail
def test_function(): ...
```

Alternatively, you can also mark a test as `XFAIL` from within the test or its setup function imperatively:

```
def test_function():
    if not valid_config():
        pytest.xfail("failing configuration (but should work)")
```

Further Reading: <https://docs.pytest.org/en/latest/how-to/skipping.html>

Example:

```
def test_health_check(client):
    response = client.get("/health-check", headers={'token': 'xyz'})
    assert response.status_code == 200
    assert b"Welcome" in response.data

def test_health_check_route_on_server(base_url):
```

```
response = requests.get(f"{base_url}/health-check", headers={'token': 'xyz'})
assert response.status_code == 200
assert b"Welcome" in response.content
```

Feature	test_health_check(client)	test_health_check_route_on_server(base_url)
Interaction Method	Test Client	HTTP Request with requests
Network Call	No	Yes
Server Dependency	Yes (assumed running)	Yes (needs to be running)
Typical Use Case	Integration Testing	End-to-End Testing

Mocking

Mocking is a technique used in testing to replace real objects or functions with simulated objects called **mocks**.

The `Client` object is a layer of abstraction that handles all of the app's interactions with the server. To isolate our code from the server, we need to somehow replace all the calls to `Client` with dummies.

The canonical way to handle this problem is by *mocking* the `Client` calls. We will replace the called functions with *mocks* that, instead of contacting the server, just return hard-coded objects that we expect to receive from the server in response to the call we expect the mocked function to receive.

Key concepts:

- **Mock object:** A simulated object that replaces a real object during testing.
- **Patching:** The process of replacing a real object with a mock object.
- **Assertions:** Verifying the interactions with the mock object.

```
import pytest
from unittest.mock import patch

def my_function(external_service):
    data = external_service.get_data()
    return process_data(data)

@patch('module_name.external_service')
```

```
def test_my_function(mock_external_service):
    mock_external_service.get_data.return_value = {'data': 'mocked_data'}
    result = my_function(mock_external_service)
    assert result == 'processed_mocked_data'
```

Handling Exceptions

- If the function handles exceptions and returns JSON responses, you don't use `pytest.raises` but instead test the response directly.
- If the function is expected to propagate exceptions, then `pytest.raises` is appropriate for ensuring the error is raised.

```
#app.py
def divide():
    try:
        data = request.json
        result = data['numerator'] / data['denominator']
        return jsonify({"result": result}), 200
    # except ZeroDivisionError:
    #     return jsonify({"error": "Division by zero"}), 400
    except KeyError:
        return jsonify({"error": "Invalid input"}), 400

#test_app.py
def test_zero_division_error(client):
    with pytest.raises(ZeroDivisionError):
        data = {"numerator": 10, "denominator": 0}
        response = client.post("/divide", json=data)
```

This will raise `ZeroDivisionError` since exception handling is not done, and hence test will pass if numerator = 10 (or any number) and denominator = 0.

But if exception handling is done separately, we can directly parse the JSON as output, using normal assert statements, as follows: (considering the `ZeroDivisionError` is uncommented above)

```
def test_divide_by_zero(client):
    data = {"numerator": 10, "denominator": 0}
    response = client.post("/divide", json=data)
    assert response.status_code == 400
    assert response.json == {"error": "Division by zero"}
```

References:

- Setting up ENV: <https://flask.palletsprojects.com/en/2.3.x/config/#TESTING>
- Temp Files and Paths: https://docs.pytest.org/en/latest/how-to/tmp_path.html
- Everything: <https://docs.pytest.org/en/latest/reference/reference.html>

- Logging: <https://pytest-with-eric.com/pytest-best-practices/pytest-logging/>
- <https://docs.pytest.org/en/latest/how-to/index.html#how-to>
- <https://werkzeug.palletsprojects.com/en/2.3.x/test/#werkzeug.test.EnvironBuilder>